

Rolling Mean-Variance Portfolio Construction and Covariance Conditioning

Tushar Chellamsetty

August 2026

Question 1: Data Description

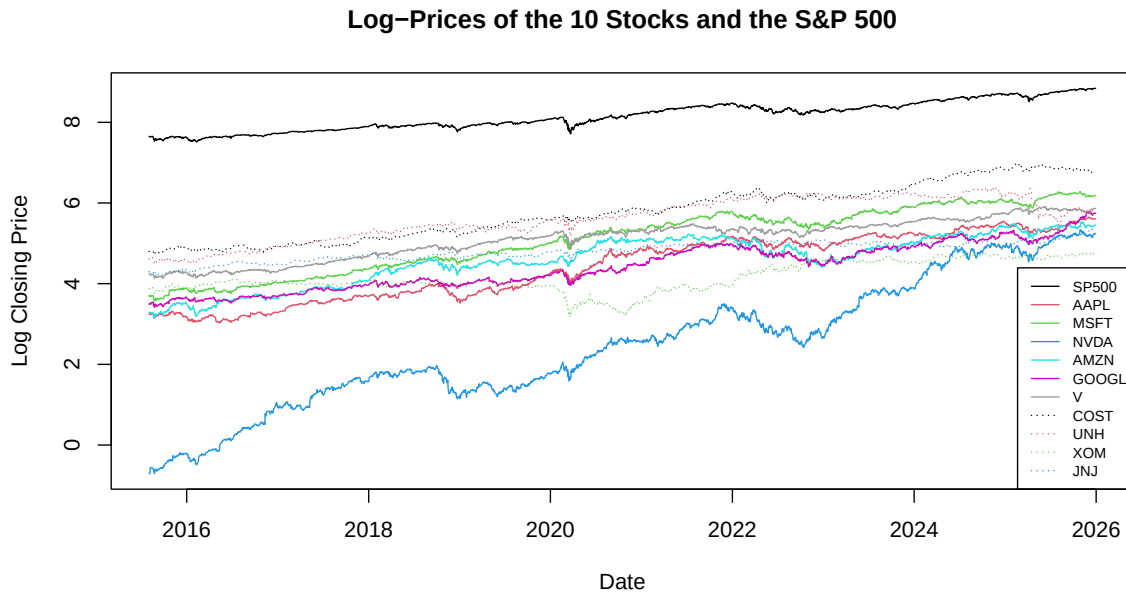


Figure 1: Log-prices of the S&P 500 and 10 selected constituent stocks.

The daily closing prices of ten S&P 500 constituents (Apple, Microsoft, Nvidia, Amazon, Alphabet, Visa, Costco, UnitedHealth, Exxon Mobil and Johnson & Johnson), together with the S&P 500 index, are downloaded from Yahoo Finance and adjusted closing prices are used throughout.

Missing values are handled by complete-case deletion: any date on which one or more of the eleven series (10 stocks and S&P500 index price) has no recorded price is dropped in full, and no price is filled in. Filling forward instead, as in Chapter 3 of the lecture notes, records a zero return for a series on a day the others move, which pulls the estimated correlations towards zero and distorts the off-diagonal entries of the covariance matrix reported below. That fill is needed there because Chapter 3 aligns ten world indices traded on different exchanges, one of which is routinely closed while another trades. The eleven series here are the S&P 500 and ten of its own constituents, all US-listed on a single trading calendar, so a missing price would be an isolated data error rather than a structural absence, and dropping the date costs almost nothing while keeping every return an observed price change.

The daily log-return of stock j on day t is the first difference of its log price,

$$r_{j,t} = \log P_{j,t} - \log P_{j,t-1}$$

where $P_{j,t}$ is the closing price of stock j on day t .

Figure 1 shows the log-prices trending upward with no constant mean and moving together over the decade; they are therefore non-stationary, and the analysis proceeds on the log-returns.

Condition Number of the Sample Covariance Matrix

Table 1: Condition number of the sample covariance matrix of daily returns, by window.

	Days	Condition number
10-year	2500	26.0
5-year	1250	22.4
1-year	250	20.5

The condition number of a matrix A is

$$\text{Cond}(A) = \frac{|\lambda|_{\max}(A)}{|\lambda|_{\min}(A)}$$

where $|\lambda|_{\max}(A)$ and $|\lambda|_{\min}(A)$ are its largest and smallest absolute eigenvalues. It is computed over the past ten, five and one years, taking 250 trading days per year (Table 1).

The condition numbers are 26, 22.4 and 20.5 for the ten-, five- and one-year windows. A large condition number has two causes: the number of variables p being close to the sample size D , and multicollinearity among the series. Here $p = 10$ is far below D in every window ($p/D = 0.04$ even for the one-year window), so the dimension effect is negligible and the conditioning comes from the covariance structure. Two features of that structure matter. First, the ten stocks move together through a common market factor; their mean off-diagonal correlation is 0.4, 0.3 and 0.24 over the three windows, moderate rather than large. Second, the individual return variances differ across the ten stocks by a factor of about 7 in the ten-year window, which also contributes to the eigenvalue spread.

These moderate values matter for Question 2. Multicollinearity shrinks the smallest eigenvalues of the sample covariance, which in turn inflates the eigenvalues of its inverse, so a badly conditioned S would make S^{-1} , and any weight that depends on it, numerically unstable. Because the condition numbers here stay moderate, that instability does not arise at the window lengths used in Question 2.

The condition number falls as the window shortens. Since p/D stays tiny throughout, and is in fact largest in the one-year window where the condition number is smallest, this movement cannot be a sample-size effect; it instead reflects the changing correlation and variance structure of returns from one period to the next.

Question 2: Rolling-Window Mean-Variance Strategy

Let r_t be the vector of daily log-returns of the $p = 10$ stocks on day t ,

$$r_t = (r_{1,t}, \dots, r_{p,t})^\top$$

with mean vector μ and covariance matrix Σ .

The strategy has four inputs: a warm-up time t_0 , an estimation window length D , a holding period I , and a target-return tuning parameter x . Given these, it proceeds in the four steps below.

(i) Estimation window

At each rebalance time t , starting from the warm-up time t_0 , the sample mean $\hat{\mu}$ and the sample covariance matrix $\hat{\Sigma}$ of the ten stocks' returns are estimated from the window of length D running from day $t - D + 1$ to day t , which at the first rebalance is $t_0 - D + 1$ to t_0 . These estimates are the inputs to step (ii).

No risk-free asset is available, so a portfolio is a fully-invested weight vector w whose entries sum to one,

$$w^\top \mathbf{1} = 1$$

where $\mathbf{1}$ is the vector of ones.

(ii) Optimal portfolio weight

For a target return μ^* the weight minimises portfolio variance subject to the budget and target-return constraints,

$$\min_w w^\top \Sigma w \quad \text{subject to} \quad w^\top \mathbf{1} = 1, \quad \mu^* \leq w^\top \mu \quad (1)$$

By the two-fund theorem the solution combines two portfolios,

$$w = (1 - \alpha) w_{mv} + \alpha w_{mkt} \quad (2)$$

$$w_{mv} = \frac{\Sigma^{-1} \mathbf{1}}{\mathbf{1}^\top \Sigma^{-1} \mathbf{1}}, \quad w_{mkt} = \frac{\Sigma^{-1} \mu}{\mathbf{1}^\top \Sigma^{-1} \mu} \quad (3)$$

$$\alpha = \frac{\mu^* - \mu^\top w_{mv}}{\mu^\top (w_{mkt} - w_{mv})} \quad (4)$$

where w_{mv} is the minimum-variance portfolio, w_{mkt} the market portfolio, and $\alpha \in \mathbb{R}$ the fraction of wealth placed in w_{mkt} . If $\mu^\top w_{mv} \geq \mu^*$ the minimum-variance portfolio already meets the target and $\alpha = 0$; more generally α is truncated to $\max(0, \alpha)$ when $\mu^\top (w_{mkt} - w_{mv}) > 0$ and to $\min(0, \alpha)$ otherwise. The target return is

$$\mu^* = 0.01 + x \left| \frac{1}{p} \sum_{j=1}^p \hat{\mu}_j \right| \quad (5)$$

with x fixed across a run: the 1% floor sets a lower bound and x controls how far above it the target sits, a larger x tilting w further toward the market portfolio. In estimation μ, Σ are replaced by $\hat{\mu}, \hat{\Sigma}$.

The target μ^* is measured on the same daily scale as the returns themselves, so the 1% floor is a 1% daily target. That is large next to the stocks' mean daily returns, which are of order 0.05% per day over the recent window, : so the constraint binds by a wide margin at every rebalance. No position or leverage limit is imposed, so the optimiser meets the target with heavily leveraged long-short weights, and the entries of Table 2 should be read as multiples of one unit of capital.

(iii) Investment and rolling

One unit of money is split by w and held for the next I days. The I -day buy-and-hold return of stock j is

$$\rho_j = \exp\left(\sum_{s=t+1}^{t+I} r_{j,s}\right) - 1$$

and the portfolio return over the period is

$$R = w^\top \rho = \sum_{j=1}^p w_j \rho_j, \quad \rho = (\rho_1, \dots, \rho_p)^\top$$

This return and the weight vector w are recorded, the window is then rolled forward I days, $t \leftarrow t + I$, and the construction repeats. If fewer than I days remain at the final rebalance, the return is taken over the days that are left, so the last period may be shorter than I . At each rebalance the estimation window and the traded block are disjoint, so no future information enters the weights.

(iv) Annualised Sharpe ratio

With N realised period returns $R_1, \dots, R_N$, and taking 250 trading days per year, the annualised Sharpe ratio is

$$\text{SR} = \sqrt{\frac{250}{I}} \frac{\bar{R}}{s_R}, \quad \bar{R} = \frac{1}{N} \sum_{k=1}^N R_k, \quad s_R = \sqrt{\frac{1}{N-1} \sum_{k=1}^N (R_k - \bar{R})^2} \quad (6)$$

This is calculated over the whole data period, using every I -day return recorded in (iii). A year contains $250/I$ investment periods, giving the annualising factor $\sqrt{250/I}$. No risk-free rate is subtracted, since no risk-free asset is available.

Worked example: the first rebalance

To make the four steps concrete, take the first rebalance of the example run, with inputs $x = 50$, $t_0 = 550$, $D = 250$ and $I = 10$:

- (i) the estimation window is the 250 days ending at day 550; over it the mean daily returns $\hat{\mu}$ range from about -2×10^{-4} to 0.004, with $\hat{\Sigma}$ the 10×10 sample covariance matrix of those returns;
- (ii) with target $\mu^* = 0.066$, the optimiser returns the weight vector in the first row of Table 2, ranging from about -24.1 to 19;
- (iii) holding those weights for the next 10 days gives a period return of $R = 1.2614$;
- (iv) repeating this over all 207 periods and annualising gives a Sharpe ratio of 0.58.

The window is then rolled forward and the same four steps repeat a further 206 times.

Implementation and Result

The strategy is coded as the function `run_strategy`, which takes the four inputs x , t_0 , D and I as arguments, so any combination can be run by changing `x_val`, `t0_val`, `D_val` and `I_val` in the Appendix. It returns the annualised Sharpe ratio, the turnover, the period returns, and the matrix of portfolio weights.

For the example inputs $x = 50$, $t_0 = 550$, $D = 250$ and $I = 10$, the strategy runs over $N = 207$ periods. Its answer is an annualised Sharpe ratio of 0.58, together with a 207×10 matrix of portfolio weights, one row per period and one column per stock; its first five rows are shown in Table 2.

Table 2: Recorded portfolio weights for each stock over the first five I-day periods of the example run ($x = 50$, $t_0 = 550$, $D = 250$, $I = 10$).

AAPL	MSFT	NVDA	AMZN	GOOGL	V	COST	UNH	XOM	JNJ
6.27	13.76	4.75	-9.06	-8.96	5.33	-5.02	19.03	-24.12	-1.00
4.08	13.31	5.96	-8.54	-11.66	3.64	-6.28	18.41	-26.76	8.83
12.56	12.61	5.51	-1.69	-16.14	4.24	-3.59	22.69	-33.85	-1.33
11.44	8.57	5.23	2.01	-15.70	7.98	-0.86	17.11	-37.29	2.51
13.54	2.12	3.19	6.81	-21.61	11.78	5.33	15.65	-43.11	7.29

Question 3: Sharpe Ratio versus Turnover

Turnover metric

Writing $w_1, \dots, w_N$ for the weight vectors at the N rebalances (one per period return R_k , so the same N as in equation (6)), the average absolute change in weights is

$$\text{AvsAchg} = \frac{1}{N} \sum_{k=1}^{N-1} \|w_{k+1} - w_k\|_1 \quad (7)$$

where the ℓ_1 norm sums the absolute entries of a vector,

$$\|v\|_1 = \sum_{j=1}^p |v_j|$$

A smaller AvsAchg is preferable for two reasons. First, each unit of $\|w_{k+1} - w_k\|_1$ is a change of position that has to be traded, and every trade carries a cost that the gross Sharpe ratio of equation (6) ignores, so a larger cumulative weight change erodes realised returns, and a large position may not be tradeable at the quoted price if the stock is not liquid enough to absorb it. Second, large weight changes indicate that the weights are chasing estimation noise in $\hat{\mu}$ and $\hat{\Sigma}$ rather than stable signal, so a low-turnover strategy is both cheaper to run and more stable.

Comparing the Sharpe ratios

The strategy is run over a grid of $x \in \{0, 10, 25, 50, 100, 200\}$, $D \in \{60, 90, 125, 180, 250, 375, 500\}$ and $I \in \{5, 10\}$, giving 84 parameter combinations, and the annualised Sharpe ratio and AvsAchg are recorded for each.

Table 3: The ten parameter combinations with the highest annualised Sharpe ratio, and their turnover (AvsAchg).

x	D	I	Sharpe	AvsAchg
10	375	10	0.76	7.1
50	375	5	0.75	15.1
100	375	5	0.75	27.8
200	375	5	0.75	53.3
25	375	5	0.75	8.7
25	375	10	0.75	12.3
0	375	10	0.75	3.9
50	375	10	0.74	21.2
10	375	5	0.74	5.0
100	375	10	0.74	38.9

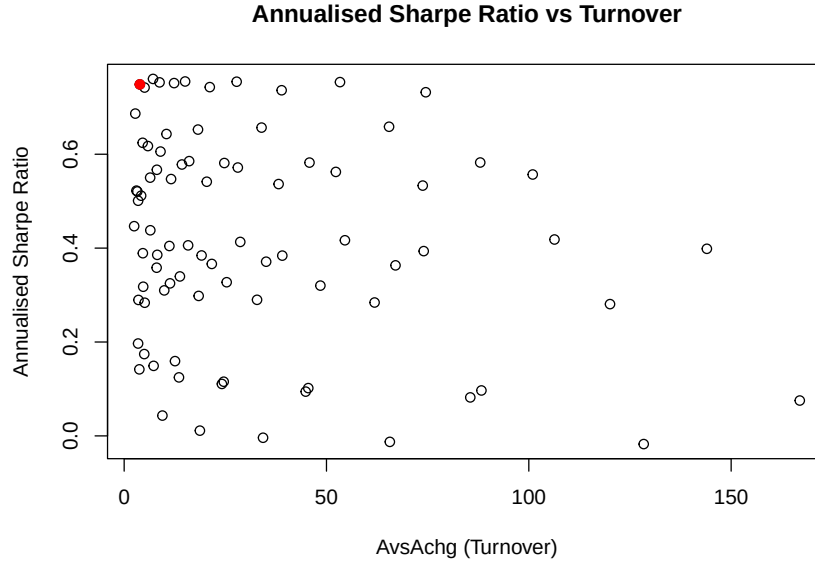


Figure 2: Annualised Sharpe ratio against turnover across all 84 settings; the chosen setting is marked in red.

The holding period makes little difference. In Figure 2 the $I = 10$ settings reach Sharpe ratios up to 0.76 against 0.75 for $I = 5$, but the medians across settings are 0.4 and 0.41 respectively, so neither dominates. The two groups overlap throughout rather than forming cleanly separated bands, and Table 3 contains settings of both kinds.

Raising the target return, by contrast, buys almost no extra Sharpe. Holding $D = 375$ and $I = 10$ fixed, the Sharpe ratio is not monotone in x : it peaks at $x = 10$ (0.76) at a turnover of 7.1, then falls back to 0.73 at $x = 200$, while AvsAchg rises steeply from 3.9 at $x = 0$ to 74.5. The best raw Sharpe therefore sits only marginally above the value of 0.75 at $x = 0$, and buying it costs a large amount of extra trading.

The estimation window matters most. At $x = 0$ and $I = 10$ the Sharpe ratio ranges from 0.17 at $D = 90$ to 0.75 at $D = 375$, and every setting in Table 3 shares that same window length. A longer window estimates $\hat{\mu}$ and $\hat{\Sigma}$ from more data and so reduces estimation error, but the framework assumes those quantities are constant over the window, which becomes less tenable the further back it reaches. The warm-up t_0 is held at 550 throughout so that every setting is evaluated over the same trading period.

Choosing a Sharpe/turnover balance

The preferred setting maximises the Sharpe ratio penalised by turnover,

$$\max_{x, D, I} \text{Sharpe} - c \cdot \text{AvsAchg}, \quad c = 0.01$$

where c is the Sharpe ratio one is willing to give up per unit of turnover. Repeating the choice at per-unit costs of 0.005, 0.01, 0.02 returns the same setting each time, $x=0$, $D=375$, $I=10$, so it is not driven by the particular value of c .

This gives $x = 0$, $D = 375$ and $I = 10$, the red point in Figure 2, with a Sharpe ratio of 0.75 at an AvsAchg of only 3.9. Table 3 lists one setting with a marginally higher Sharpe ratio and several matching it at far greater turnover; the chosen setting; the chosen setting gives up only 0.01 of Sharpe against the highest in the grid, which is the better trade-off once trading costs are considered.

The reason a low target return wins the balance is estimation error. Only the market fund uses the sample mean $\hat{\mu}$, the noisiest quantity to estimate, while the minimum-variance fund in (3) ignores it entirely.

Raising x loads more on the $\hat{\mu}$ -dependent fund: the maximum absolute weight grows from about 6 at $x = 0$ to over 128 at $x = 200$, and turnover from 3.9 to 74.5, yet the Sharpe ratio barely moves. Among the settings tested, $x = 0$ places the least weight on the noisy market fund, since α is smallest when the target is lowest.

Conclusion

The setting $x = 0$, $D = 375$, $I = 10$ gives the best balance of risk-adjusted return and tradeability. The estimation window is the parameter that matters, performance improving sharply with D up to 375; the holding period makes little difference, and raising the target return through x adds large turnover for at most a marginal gain in Sharpe ratio.

Appendix: R Code

```
# Packages
knitr::opts_chunk$set(echo = FALSE, message = FALSE, warning = FALSE,
                      fig.align = "center")

library(knitr)
library(quantmod)
# download prices
tickers <- c("AAPL", "MSFT", "NVDA", "AMZN", "GOOGL", "V", "COST", "UNH", "XOM", "JNJ")
getSymbols(c("^GSPC", tickers), from = "2015-08-01", to = "2025-12-31")

# Column 6 is adjusted close
prices <- merge(GSPC[, 6], AAPL[, 6], MSFT[, 6], NVDA[, 6], AMZN[, 6],
               GOOGL[, 6], V[, 6], COST[, 6], UNH[, 6], XOM[, 6], JNJ[, 6])
colnames(prices) <- c("SP500", tickers)

#Missing values handled
keep <- rep(TRUE, dim(prices)[1])
for (j in 1:ncol(prices)) {
  keep <- keep & !is.na(as.vector(prices[, j]))
}

n_dropped <- sum(!keep)
prices <- prices[keep, ]

# Log-prices for the plot

dates <- index(prices)
lp <- log(as.matrix(prices))
ret <- diff(log(as.matrix(prices[, tickers])))
# log-price plot
cols <- c(1:6, 8, 1:4)
ltys <- c(rep(1, 7), rep(3, 4))

plot(dates, lp[, 1], type = "l", col = cols[1], lty = ltys[1],
     ylim = c(min(lp), max(lp)),
     xlab = "Date", ylab = "Log Closing Price",
     main = "Log-Prices of the 10 Stocks and the S&P 500")
for (j in 2:ncol(lp)) {
  lines(dates, lp[, j], col = cols[j], lty = ltys[j])
}
legend("bottomright", colnames(lp), col = cols, lty = ltys,
      ncol = 1, cex = 0.7)
#mean of the off-diagonal correlations
mean_offdiag_cor <- function(R) {
  C <- cor(R)
  (sum(C) - ncol(C)) / (ncol(C)^2 - ncol(C))
}

# Condition number calculated
condition_number <- function(S) {
  ev <- abs(eigen(S, symmetric = TRUE, only.values = TRUE)$values)
  max(ev) / min(ev)
}
```

```

}

# Calculated over 3 periods
windows <- c("10-year" = 2500, "5-year" = 1250, "1-year" = 250)
nret <- dim(ret)[1]

cond_tab <- numeric(3)
corr_tab <- numeric(3)
for (i in 1:3) {
  R_win <- ret[(nret - windows[i] + 1):nret, ]
  cond_tab[i] <- condition_number(var(R_win))
  corr_tab[i] <- mean_offdiag_cor(R_win)
}

# Direction across the windows
cond_dir <- if (cond_tab[3] < cond_tab[1]) "falls" else
  if (cond_tab[3] > cond_tab[1]) "rises" else "is roughly unchanged"

# Var spread
vars <- diag(var(ret[(nret - windows[1] + 1):nret, ]))
var_ratio <- max(vars) / min(vars)

# Typical daily mean return
recent <- ret[(nret - 249):nret, ]
mu_recent <- numeric(ncol(recent))
for (j in 1:ncol(recent)) {
  mu_recent[j] <- mean(recent[, j])
}
mu_daily <- abs(mean(mu_recent))

# Condition-number table
cond_out <- matrix(0, 3, 2)
rownames(cond_out) <- names(windows)
colnames(cond_out) <- c("Days", "Condition number")
cond_out[, "Days"] <- windows
cond_out[, "Condition number"] <- round(cond_tab, 1)
kable(cond_out, digits = c(0, 1),
  caption = paste("Condition number of the sample covariance",
    "matrix of daily returns, by window."))

# Two-fund optimiser
opt_portfolio <- function(mu, Sigma, mu_star) {
  ones <- rep(1, length(mu))
  Sinv <- solve(Sigma)

  w_mv <- (Sinv %*% ones) / as.numeric(t(ones) %*% Sinv %*% ones)
  w_mkt <- (Sinv %*% mu) / as.numeric(t(ones) %*% Sinv %*% mu)

  den <- as.numeric(t(mu) %*% (w_mkt - w_mv))
  a <- (mu_star - as.numeric(t(mu) %*% w_mv)) / den

  a <- if (den > 0) max(0, a) else min(0, a)
}

```

```

    as.numeric((1 - a) * w_mv + a * w_mkt)
}

# Roll the strategy across the sample
run_strategy <- function(R, x, t0, D, I) {
  Tn <- dim(R)[1]
  reb <- seq(from = t0, to = Tn - 1, by = I)
  m <- length(reb)

  period_ret <- rep(0, m)
  W <- matrix(0, m, ncol(R))

  for (k in 1:m) {
    t <- reb[k]

    # estimate mean returns and covariance from the past D days.
    est <- R[(t - D + 1):t, ]
    mu <- numeric(ncol(est))
    for (j in 1:ncol(est)) {
      mu[j] <- mean(est[, j])
    }
    Sig <- var(est)

    # weights for the target return
    w <- opt_portfolio(mu, Sig, 0.01 + x * abs(mean(mu)))

    #hold for I days
    hold <- (t + 1):min(t + I, Tn)
    simI <- numeric(ncol(R))
    for (j in 1:ncol(R)) {
      simI[j] <- exp(sum(R[hold, j])) - 1
    }

    #record return and weights
    period_ret[k] <- sum(w * simI)
    W[k, ] <- w
  }

  # Annualised Sharpe ratio
  sharpe <- sqrt(250 / I) * mean(period_ret) / sqrt(var(period_ret))

  # Average absolute weight change
  total_change <- 0
  for (k in 2:m) {
    total_change <- total_change + sum(abs(W[k, ] - W[k - 1, ]))
  }
  avsachg <- total_change / m

  list(sharpe = sharpe, avsachg = avsachg,
       period_ret = period_ret, weights = W)
}

# Strategy inputs

```

```

x_val <- 50      # target-return
t0_val <- 550    # warm-up time
D_val <- 250    # estimation window
I_val <- 10     # holding period

ex <- run_strategy(ret, x = x_val, t0 = t0_val, D = D_val, I = I_val)

# First rebalance of this run,
est1 <- ret[(t0_val - D_val + 1):t0_val, ]
mu1 <- numeric(ncol(est1))
for (j in 1:ncol(est1)) {
  mu1[j] <- mean(est1[, j])
}
mustar1 <- 0.01 + x_val * abs(mean(mu1)) # target return
w1 <- ex$weights[1, ]                  # weight vector
r1 <- ex$period_ret[1]                  # I-day portfolio return
# 1st 5 weight vectors
w5 <- round(ex$weights[1:5, ], 2)
colnames(w5) <- tickers
kable(w5,
      caption = paste0("Recorded portfolio weights for each stock over the first five I-day ",
                        "periods of the example run (x = ", x_val, ", t0 = ", t0_val,
                        ", D = ", D_val, ", I = ", I_val, ")."))

# Parameter grid
x_grid <- c(0, 10, 25, 50, 100, 200)
D_grid <- c(60, 90, 125, 180, 250, 375, 500)
I_grid <- c(5, 10)
n_grid <- length(x_grid) * length(D_grid) * length(I_grid)

grid <- matrix(0, n_grid, 5)
colnames(grid) <- c("x", "D", "I", "Sharpe", "AvsAchg")

i <- 0
for (I in I_grid) {
  for (D in D_grid) {
    for (x in x_grid) {
      i <- i + 1
      t0 <- 550
      res <- run_strategy(ret, x, t0, D, I)

      grid[i, "x"] <- x
      grid[i, "D"] <- D
      grid[i, "I"] <- I
      grid[i, "Sharpe"] <- res$sharpe
      grid[i, "AvsAchg"] <- res$avsachg
    }
  }
}

#best by turnover-penalised Sharpe.
obj <- grid[, "Sharpe"] - 0.01 * grid[, "AvsAchg"]
best <- grid[which(obj == max(obj))[1], ]

```

```

# Sharpe ranges for the two holding periods, for the test.
sr10 <- c(min(grid[grid[, "I"] == 10, "Sharpe"]),
          max(grid[grid[, "I"] == 10, "Sharpe"]))
sr5  <- c(min(grid[grid[, "I"] == 5, "Sharpe"]),
          max(grid[grid[, "I"] == 5, "Sharpe"]))

med10 <- median(grid[grid[, "I"] == 10, "Sharpe"])
med5  <- median(grid[grid[, "I"] == 5, "Sharpe"])

# Effect of x at the winning D and I, and the peak within that slice.
xe    <- grid[grid[, "D"] == best["D"] & grid[, "I"] == best["I"], ]
s0    <- xe[xe[, "x"] == min(x_grid), "Sharpe"]
s200  <- xe[xe[, "x"] == max(x_grid), "Sharpe"]
a0    <- xe[xe[, "x"] == min(x_grid), "AvsAchg"]
a200  <- xe[xe[, "x"] == max(x_grid), "AvsAchg"]
xe_best <- xe[which(xe[, "Sharpe"] == max(xe[, "Sharpe"]))[1], ]

sharpe_gap <- max(grid[, "Sharpe"]) - best["Sharpe"]

# Which setting wins the balance under a few different trading costs.
costs    <- c(0.005, 0.01, 0.02)
pen_check <- rep("", length(costs))
for (j in 1:length(costs)) {
  objc <- grid[, "Sharpe"] - costs[j] * grid[, "AvsAchg"]
  b    <- grid[which(objc == max(objc))[1], ]
  pen_check[j] <- paste0("x=", b["x"], ", D=", b["D"], ", I=", b["I"])
}

pen_uniq <- unique(pen_check)

# Ten highest-Sharpe rows, found by taking the maximum ten times over.
ranked <- grid[, "Sharpe"]
top_ind <- numeric(10)
for (k in 1:10) {
  idx <- which(ranked == max(ranked))[1]
  top_ind[k] <- idx
  ranked[idx] <- -Inf
}
top <- grid[top_ind, c("x", "D", "I", "Sharpe", "AvsAchg")]

kable(top, row.names = FALSE, digits = c(0, 0, 0, 2, 1),
      caption = paste("The ten parameter combinations with the highest",
                      "annualised Sharpe ratio, and their turnover",
                      "(AvsAchg)."))

de    <- grid[grid[, "x"] == best["x"] & grid[, "I"] == best["I"], ]
de_min <- de[which(de[, "Sharpe"] == min(de[, "Sharpe"]))[1], ]
de_max <- de[which(de[, "Sharpe"] == max(de[, "Sharpe"]))[1], ]

# Sharpe versus turnover
plot(grid[, "AvsAchg"], grid[, "Sharpe"],
     xlab = "AvsAchg (Turnover)", ylab = "Annualised Sharpe Ratio",

```

```
    main = "Annualised Sharpe Ratio vs Turnover")
points(best["AvsAchg"], best["Sharpe"], col = "red", pch = 19)
# Largest weight at x extremes
t0b <- 550
wmax0 <- max(abs(run_strategy(ret, min(x_grid), t0b, best["D"], best["I"])$weights))
wmax200 <- max(abs(run_strategy(ret, max(x_grid), t0b, best["D"], best["I"])$weights))
```